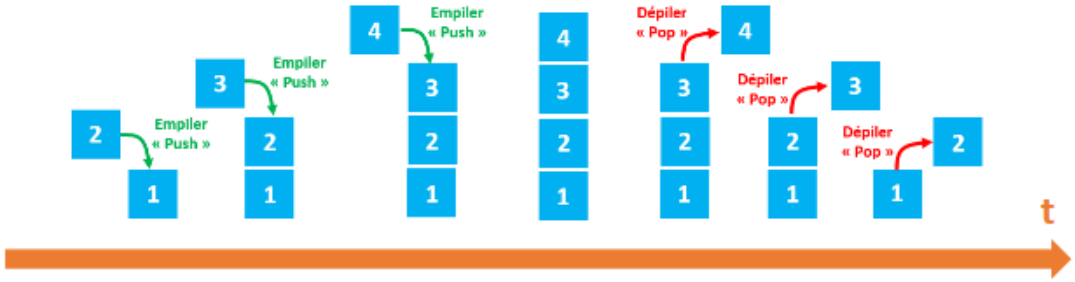
SD02 – Listes Piles Files

1. Les piles

Définition 1: Les piles – Stack

Une **pile** est un **Type Abstrait de Données (TAD)** qui fonctionne selon le principe du *dernier arrivé, premier sorti.* (Last In, First Out: LIFO). C'est le principe d'une pile d'assiettes: la dernière ajoutée au sommet de la pile sera la première à être utilisée.

Exemple 1:




```
class Pile:
******************
pile (list)
nb_elements (int)
taille_pile (int)
******************
creer_pile(pile, taille_pile)
empiler(pile, element)
depiler(pile) -> element
est_vide(pile) -> bool
est_pleine(pile) -> bool
```

Exercice 1: Implémenter en python la classe `Pile`

Code Python

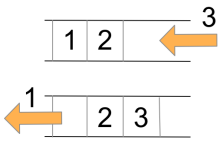
```python
class Pile:
    def __init__(self, taille = None):
        ''' Si taille est None, la pile n'a pas de taille maximum '''
        ...

    def est_vide(self):
        ''' Renvoie True si la pile est vide '''
    def est_pleine(self):
        ''' Renvoie True si la pile est pleine '''
    def empiler(self, element):
        ''' Empile element si non pleine '''
    def depiler(self):
        ''' Renvoie l'élément dépilé si non vide '''
```

2. Les files

Définition 2: Les files – Queue

Une **pile** est un **Type Abstrait de Données (TAD)** qui fonctionne selon le principe du *premier arrivé, premier sorti.* (First In, First Out: FIFO). C'est le principe d'une file d'attente. On utilise ce type de données pour par exemple la gestion des stocks dans un entrepôt (pour des raison de date de consommation) ou pour la file d'attente d'impression.



Exercice 2:

```
class File:
*****************
file (list)
nb_elements (int)
taille_file (int)
*****************
creer_file(file, taille_file)
enfiler(file, element)
defiler(file) -> element
est_vide(file) -> bool
est_pleine(pile) -> bool
```

Code Python

```python
class File:
    def __init__(self, taille = None):
        ''' Si taille est None, la file n'a pas de taille maximum '''
        ...

    def est_vide(self):
        ''' Renvoie True si la file est vide '''
    def est_pleine(self):
        ''' Renvoie True si la file est pleine '''
    def enfiler(self, element):
        ''' Enfile element si non pleine '''
    def defiler(self):
        ''' Renvoie l'élément défilé si non vide '''
```

3. Implémentation du TAD à partir d'une classe Maillon

Définition 3: Le maillon

Il est possible de définir une structure de données abstraire sans connaitre l'intégralité de la structure.

Exemple 2: La file chainée

```
class Maillon:
*************************
nom (int)
suivant (Maillon ou None)
*************************
creer_maillon(maillon)
get_suivant(maillon)
get_nom(maillon)
set_suivant(maillon, maillon_suivant)
afficher_maillon(maillon)

class File_chainee:
*************************
premier_element (Maillon ou None)
*************************
creer_file_chainee(file_chainee)
est_vide(file_chainee)
enfiler(file_chainee, maillon)
defiler(file_chainee)
afficher_file(file_chainee)
```

Code Python

```python
class Maillon:
    def __init__(self, valeur, suivant = None):
        self.valeur = valeur
        self.suivant = suivant          # de type Maillon

class File_chainee:
    def __init__(self, tete = None):
        self.tete = tete                # de type Maillon
    def est_vide(self):
        return self.tete == None
    def enfiler(self, valeur):
        maillon_a_inserer = Maillon(valeur)
        ...
    def defiler(self):
        ...
    def __str__(self):
        if not self.tete:
            return f'None'
        liste = []
        courant = self.tete
        while courant:
            liste.append(str(courant.valeur))
            courant = courant.suivant
        elements = "<-".join(liste)
        return elements
```

4. Implémentation d'une file par un tableau circulaire

L'implémentation d'une file est couteuse si la file est grande puisque qu'à chaque retrait, il faut supprimer un élément et décaler tous les autres d'un indice. Une implémentation plus efficace consiste à considérer le tableau comme un anneau dont la file occupe les éléments de l'indice `debut` à l'indice `fin - 1`.

La file a donc ici une taille maximum.

- Lorsqu'on enfile, on ajoute l'élément à l'indice `fin`
- Lorsqu'on doit enfiler un élément et qu'on a atteint la fin du tableau, on continue à le remplir au début si le tableau n'est pas plein.
- Lorsqu'on retire un élément, on retire celui de l'indice `debut` et lorsqu'on arrive à la fin du tableau au remet `debut` à `0`.
- Les indice `debut` et `fin` vont donc varier au fur et à mesure qu'on ajoute ou qu'on supprimer des éléments.

En réalité, les éléments ne sont pas vraiment supprimés, ce sont les indices `debut` et `fin` qui délimitent la file.

